Car Collision Detection System

A Prototype for Automotive Safety Using Arduino

Name/Class/Roll no: Nitish Bhosle/ D15A/ 04
Name/Class/Roll no: Aryan Dangat/ D15A/12

Date: 17/04/25
Subject: Wireless Technology (WT)

Table of Contents

Introduction

This report presents the design and implementation of a Car Collision Detection System developed with an Arduino Uno microcontroller. The primary objective of the system is to enhance vehicle safety by detecting sudden impacts or movements that may indicate a collision, while simultaneously displaying the vehicle's location data on an LCD screen. The key components of this system include an accelerometer and gyroscope for collision detection, a NEO-6M GPS module for precise location tracking, and an I2C LCD for real-time status updates. Prototyping is facilitated through the use of a breadboard, which allows for flexible connections. This project exemplifies the application of embedded systems in automotive safety by integrating sensor data acquisition, location tracking, and user interface design into a cohesive functional prototype.

Components

The system is comprised of five essential components, each playing a vital role in its overall functionality. The following is a detailed enumeration of these components along with their respective functions:

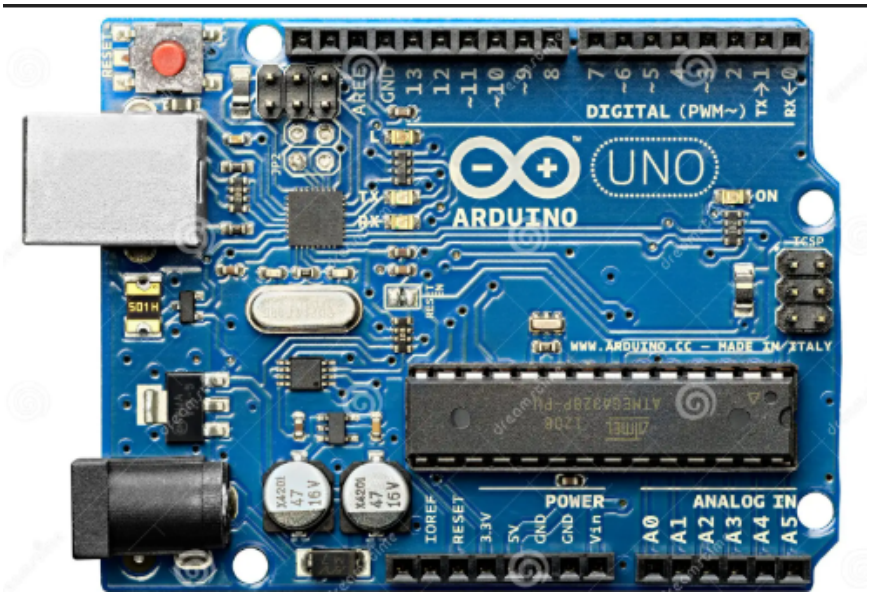
- **Arduino Uno:** The main microcontroller that processes sensor data and coordinates module communication.

  The Arduino Uno serves as the central "brain" of the car collision detection system. It's a microcontroller board that processes data from the MPU6050 (motion sensor) and NEO-6M GPS module, determines whether a collision has occurred, and sends updates to the I2C LCD. The Arduino reads acceleration data from the MPU6050 to detect sudden spikes that might indicate a crash. If a collision is confirmed, it retrieves location data from the NEO-6M and displays both the status and coordinates on the LCD. It uses digital and analog input/output (I/O) pins to interface with these components and runs a programmed algorithm to coordinate their operation in real time.

  Metrics:

  - **Operating Voltage:** 5V
  - **Clock Speed:** 16 MHz (fast enough for real-time sensor processing)
  - **Digital I/O Pins:** 14 (used for serial communication with the GPS module)
  - **Analog Input Pins:** 6 (not used in this project)
  - **Communication Protocols:** I2C (for MPU6050 and LCD), Serial (for NEO-6M and debugging)

  The Arduino Uno's ability to handle multiple communication protocols and process data quickly makes it ideal for managing this system.



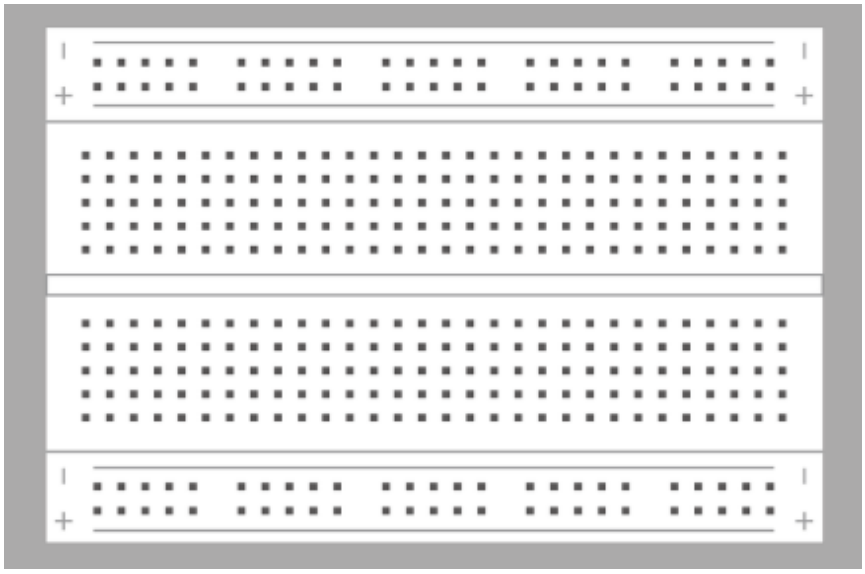
- **Breadboard:** Provides a solder-free platform for prototyping and connecting components.

  The breadboard provides a flexible, solder-free platform for connecting all the components in the system. It has rows of holes that are internally wired together, allowing you to plug in the Arduino, MPU6050, NEO-6M, and I2C LCD, then link them with jumper wires. In this project, it distributes power (5V and ground) from the Arduino to the other modules and facilitates signal connections, making it easy to assemble and modify the circuit during development.

  Metrics:

  - **Current Rating:** Designed for low-current applications (suitable for sensors and microcontrollers)
  - **Hole Spacing:** Standard 0.1-inch (2.54 mm) pitch (matches common component pin spacing)

  While the breadboard doesn't have advanced metrics, its simplicity and adaptability are key for prototyping this collision detection system.



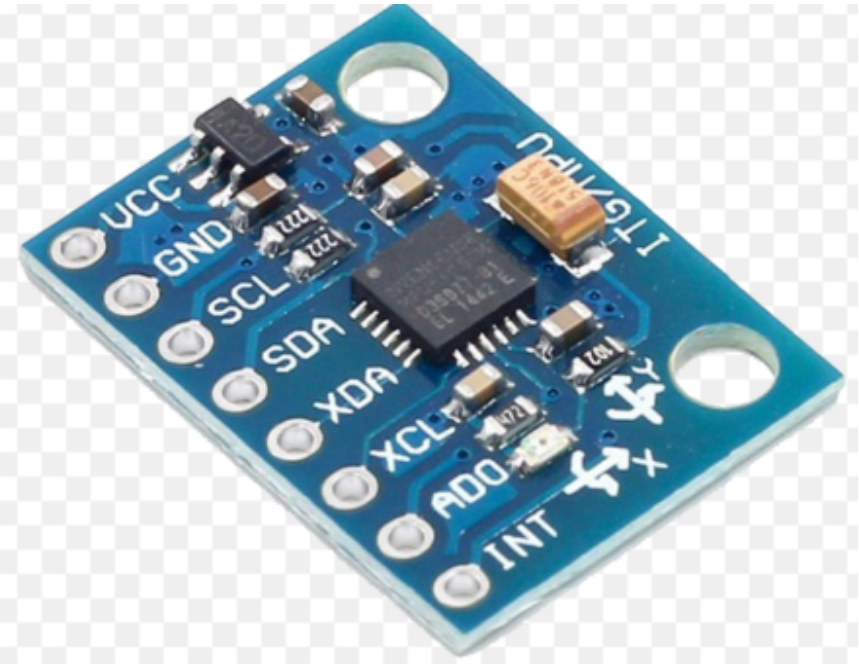
- **MPU6050 Accelerometer & Gyroscope:** A 6-axis motion sensor that detects sudden changes in motion, signaling a potential collision.

  The MPU6050 is a 6-axis motion sensor that combines a 3-axis accelerometer and a 3-axis gyroscope. The accelerometer measures linear acceleration (e.g., changes in speed), while the gyroscope tracks angular velocity (e.g., rotation). In this collision detection system, the accelerometer is the primary focus—it detects sudden changes in motion, such as a rapid deceleration or impact, which could signal a collision. The Arduino reads this data via the I2C protocol, calculates the acceleration magnitude, and compares it to a threshold to decide if a crash has occurred.

  Metrics:

  - **Accelerometer Range:** ±2g to ±16g (configurable; ±8g is typical for collision detection)
  - **Gyroscope Range:** ±250°/s to ±2000°/s (configurable; not critical for this project)
  - **Operating Voltage:** 3.3V to 5V
  - **Communication Protocol:** I2C (default address: 0x68)
  - **Sensitivity:** Varies with range (e.g., 4096 LSB/g at ±8g)

  The MPU6050's high sensitivity to acceleration changes ensures reliable detection of potential collisions.



- **NEO-6M GPS Module:** Retrieves precise location data from GPS satellites.

  The NEO-6M GPS module retrieves the vehicle's precise location by receiving signals from GPS satellites. It calculates latitude and longitude, then sends this data to the Arduino in NMEA format via serial communication. The Arduino parses the NMEA sentences (using a library like TinyGPS++) to extract coordinates, which are displayed on the LCD if a collision is detected. An external antenna can enhance signal reception, especially inside a vehicle.

  Metrics:

  - **Position Accuracy:** ~2.5 meters (under open sky conditions)
  - **Update Rate:** 1 Hz (updates once per second)
  - **Operating Voltage:** 3.3V to 5V
  - **Communication Protocol:** Serial (default baud rate: 9600)
  - **Time to First Fix (TTFF):** Cold start ~27 seconds, hot start ~1 second

  The NEO-6M's accuracy and consistent updates make it effective for providing location data in an emergency.

- **I2C LCD Display:** Displays system status and collision alerts with location coordinates.

  The I2C LCD is a 16x2 character display (16 columns, 2 rows) that shows system status and collision alerts. It communicates with the Arduino using the I2C protocol, which requires only two data lines (SDA and SCL), plus power and ground. During normal operation, it might display "System Ready." If a collision is detected, it shows "Collision Detected" along with the GPS coordinates from the NEO-6M. The I2C interface simplifies wiring by sharing the bus with the MPU6050.

  Metrics:

  - **Display Size:** 16 characters x 2 lines (32 characters total)
  - **Operating Voltage:** 5V
  - **Communication Protocol:** I2C (typical address: 0x27)
  - **Backlight:** Included (improves readability in low light)

  The I2C LCD's compact size and efficient communication make it a practical choice for displaying critical information.

System Design

The system integrates its components with the Arduino Uno as the central hub. Power is distributed via the breadboard, with the Arduino's 5V and GND pins supplying the MPU6050, NEO-6M GPS module, and I2C LCD. Data connections are as follows:

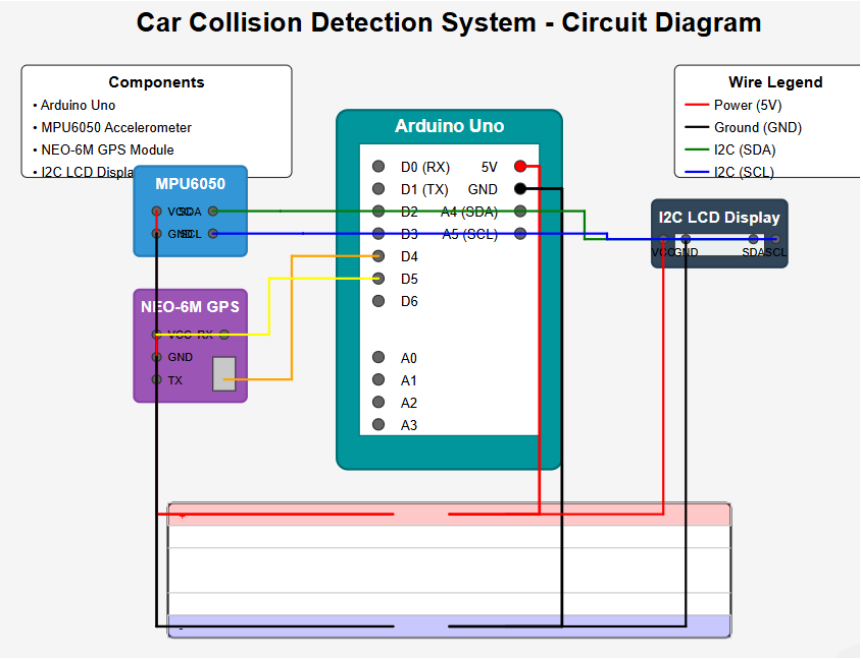
- The MPU6050 and I2C LCD use the I2C protocol, connecting to Arduino pins SDA (A4) and SCL (A5).
- The NEO-6M GPS module employs serial communication, with its TX and RX pins connected to Arduino digital pins D4 and D5, using the SoftwareSerial library to emulate a serial port.

This configuration ensures efficient data exchange, enabling real-time collision detection and location display.

Circuit Diagram

The circuit diagram visually represents the physical connections:

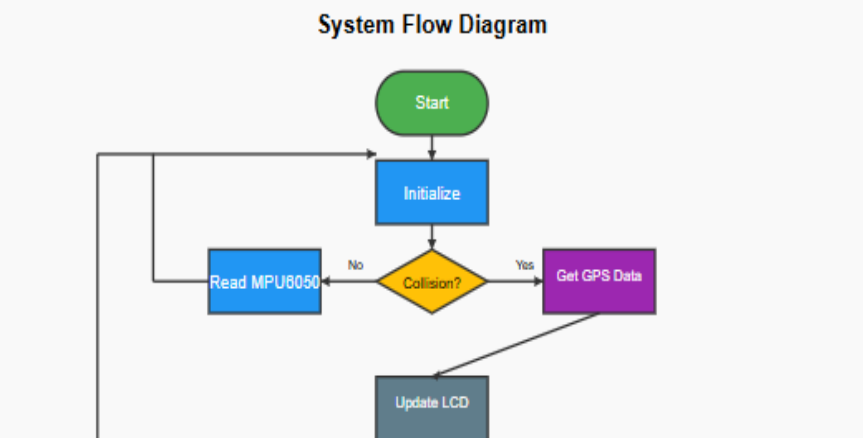
- The Arduino Uno's 5V and GND pins connect to the breadboard's power rails.
- The NEO-6M GPS module's VCC and GND pins tie to the 5V and GND rails, with TX to D4 and RX to D5.
- The MPU6050's VCC and GND connect to the power rails, with SDA to A4 and SCL to A5.
- The I2C LCD display shares the I2C bus (SDA to A4, SCL to A5) and connects to 5V and GND.



Block Diagram

The block diagram illustrates the system architecture:

- The Arduino Uno serves as the central processing unit.
- It receives input from the MPU6050 (collision detection) and NEO-6M GPS module (location data).
- Processed data is output to the I2C LCD display.
- Data flow is indicated by arrows: I2C for MPU6050 and LCD, serial for the GPS module.



Code Description

The Arduino code continuously monitors vehicle motion, detects collisions, and displays location data. It initializes the MPU6050, NEO-6M GPS module, and I2C LCD on startup. In the main loop:

- Acceleration data from the MPU6050 is read, and its magnitude is calculated.
- If the magnitude exceeds a threshold, a collision is detected, triggering GPS data retrieval and LCD updates with "Collision Detected" and coordinates.
- Otherwise, the LCDs "System Ready."

Required libraries include:

- **`Wire`** (I2C communication)
- **`Adafruit_MPU6050`** (sensor handling)
- **`SoftwareSerial`** (GPS communication)
- **`TinyGPS++`** (GPS data parsing)
- **`LiquidCrystal_I2C`** (LCD control)

Arduino Code

Below is the complete Arduino code implementing the system's functionality:

```
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <SoftwareSerial.h>
#include <TinyGPS++.h>
#include <LiquidCrystal_I2C.h>

// Initialize MPU6050
Adafruit_MPU6050 mpu;

// Initialize GPS (RX, TX pins)
SoftwareSerial gpsSerial(4, 5);
TinyGPSPlus gps;

// Initialize LCD (address 0x27, 16x2 display)
LiquidCrystal_I2C lcd(0x27, 16, 2);

// Collision detection threshold (m/s^2)
const float ACCEL_THRESHOLD = 20.0;

void setup() {
 // Start serial communication for debugging
 Serial.begin(9600);
 // Initialize I2C and LCD
 Wire.begin();
 lcd.begin();
 lcd.backlight();
 lcd.setCursor(0, 0);
 lcd.print("System Ready");

// Initialize MPU6050
 if (!mpu.begin()) {
 Serial.println("Failed to find MPU6050");
 lcd.setCursor(0, 1);
 lcd.print("MPU6050 Error");
 while (1);
 }
 mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
 mpu.setGyroRange(MPU6050_RANGE_500_DEG);
 mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
```

```
// Initialize GPS serial
 gpsSerial.begin(9600);
}

void loop() {
 // Read MPU6050 data
 sensors_event_t a, g, temp;
 mpu.getEvent(&a, &g, &temp);

// Calculate acceleration magnitude
 float accelMag = sqrt(pow(a.acceleration.x, 2) +
pow(a.acceleration.y, 2) + pow(a.acceleration.z, 2));

// Check for collision
 if (accelMag > ACCEL_THRESHOLD) {
 lcd.clear();
 lcd.setCursor(0, 0);
 lcd.print("Collision!");
 // Get GPS data
 while (gpsSerial.available() > 0) {
 gps.encode(gpsSerial.read());
 }
 if (gps.location.isValid()) {
 lcd.setCursor(0, 1);
 lcd.print("Lat: ");
 lcd.print(gps.location.lat(), 6);
 lcd.setCursor(0, 2);
 lcd.print("Lon: ");
 lcd.print(gps.location.lng(), 6);
 } else {
 lcd.setCursor(0, 1);
 lcd.print("No GPS Signal");
 }
 delay(5000); // Display for 5 seconds
 } else {
 lcd.clear();
 lcd.setCursor(0, 0);
 lcd.print("System Ready");
 }
 delay(100); // Loop delay
}
```

Explanation: The code initializes all components, reads acceleration data, and compares it to a threshold. On collision detection, it fetches GPS coordinates and updates the LCD; otherwise, it maintains a "System Ready" status.

Conclusion

This Car Collision Detection System demonstrates the synergy of sensors, a GPS module, and a microcontroller to enhance automotive safety. By leveraging the MPU6050 for collision detection and the NEO-6M for location data, displayed via the I2C LCD, it provides a practical prototype for real-time monitoring. Its modular design supports future enhancements, such as audible alerts or data logging, making it a solid foundation for advanced vehicular safety applications.

References

- Adafruit MPU6050 Library Documentation
- TinyGPS++ Library Documentation
- LiquidCrystal_I2C Library Documentation